

Post-ASR Correction for Low-Resource Rajasthani Language

ABHISHEK BHANDARI, Computer Science and Engineering, Indian Institute of Technology Jodhpur, Jodhpur, India

GAURAV HARIT, Computer Science and Engineering, Indian Institute of Technology Jodhpur, Jodhpur, India

State-of-the-art multilingual Automatic Speech Recognition (ASR) models produce systematic errors when applied to low-resource languages like Rajasthani, for which they lack dedicated training data. This article addresses this challenge by introducing a post-ASR correction framework that leverages the complementary error patterns in the outputs (termed as views) from two distinct models: Whisper-large-v3 and MMS-1B-All. We propose a multi-view,¹ character-level sequence-to-sequence (Seq2Seq) model that uses a gated fusion mechanism to dynamically weigh information from the two ASR outputs. On a new benchmark created from the IndicTTS Rajasthani corpus, our gated model achieves a Character Error Rate (CER) of 7.86% and a Word Error Rate (WER) of 29.98%. This outperforms the best single-view baselines (8.01% CER and 30.33% WER), simple multi-view concatenation (8.21% CER and 30.05% WER), as well as Llama-3.2-3B and mBART-50-large, both fine-tuned on Whisper and MMS inputs. It also surpasses powerful Large Language Models (LLMs) like GPT-4o and Gemini 2.5 Pro in a zero-shot setting. This work establishes the first baseline for post-ASR correction in Rajasthani, demonstrating that a compact, specialized model is more effective than general-purpose LLMs for this targeted, low-resource task.

CCS Concepts: • **Computing methodologies** → **Natural language processing**; **Speech recognition**;

Additional Key Words and Phrases: Automatic speech recognition, Post-ASR correction, large language models

ACM Reference Format:

Abhishek Bhandari and Gaurav Harit. 2026. Post-ASR Correction for Low-Resource Rajasthani Language. *ACM Trans. Asian Low-Resour. Lang. Inf. Process.* 25, 3, Article 15 (February 2026), 13 pages. <https://doi.org/10.1145/3793254>

1 Introduction

Low-resource speech technologies remain markedly behind high-resource counterparts, even within the same language family. For many Indic languages, limited labeled speech data and orthographic variation induce high error rates in **automatic speech recognition (ASR)**. In such settings, we reduce ASR errors with post-ASR correction: a text-only model takes first-pass transcripts from the recognizer(s) and rewrites them into the intended transcription.

¹In this context, “multi-view” refers to the approach of using the outputs from two different speech recognition models as separate “views” or perspectives on the same audio.

Authors’ Contact Information: Abhishek Bhandari (corresponding author), Computer Science and Engineering, Indian Institute of Technology Jodhpur, Jodhpur, Rajasthan, India; e-mail: bhandari.1@iitj.ac.in; Gaurav Harit, Computer Science and Engineering, Indian Institute of Technology Jodhpur, Jodhpur, Rajasthan, India; e-mail: gharit@iitj.ac.in.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2375-4699/2026/02-ART15

<https://doi.org/10.1145/3793254>

This article focuses on post-ASR correction for **Rajasthani**, an Indo-Aryan macrolanguage spoken by over 50 million people. Commonly written in the Devanagari script, it shares lexical overlap with Hindi but is often overlooked in large-scale ASR development efforts. Lacking dedicated, publicly available ASR models, users must rely on large multilingual systems such as Whisper [16] and MMS [15]. While powerful, their outputs are prone to systematic errors, underscoring the need for a reliable correction layer.

The central question we investigate is whether we can effectively correct these errors by leveraging outputs from multiple heterogeneous ASR systems. The error patterns of different models are often complementary; one may correctly transcribe a word that the other misses. We propose a **multi-view Seq2Seq model** that takes ASR hypotheses from Whisper and MMS as parallel inputs. To effectively combine these views, we introduce a lightweight, **character-level gated fusion mechanism** that enables the model to dynamically select the most reliable source at each time step.

Why character level fusion? In Devanagari, many ASR discrepancies are local and orthographic, for example, vowel matras and aspiration. As shown in Table 3, MMS yields “दिना” while Whisper yields “दिनाँ”, and “वेरियो” in place of “व्हे रियो”. Word boundary merges like “आयोगने” versus “आयोग नै” create character-level insertions and deletions near spaces. Aligning at the character level with a symbol ($\emptyset$) lets the model view these micro differences in parallel. The gated fusion can then give more weight to the view that preserves the correct matra, aspiration, or boundary at that position, which word-level combiners often miss. Our approach differs from classical system combination methods like ROVER [5], which align and vote across recognizer outputs at the word level. Such methods can be brittle in low-resource settings and may not fully exploit character-level regularities. In contrast, our formulation learns to weight the contributions of each view at a fine-grained, per-character level, allowing it to capture subtle evidence (e.g., matra placement) that is critical to the Devanagari script.

In this work, our contributions are as follows:

- We present the first systematic study of post-ASR correction for the Rajasthani language, establishing a new benchmark for the task.
- We introduce a single-encoder, dual-view architecture with a character-wise gated fusion module designed to efficiently leverage complementary error patterns in the inputs ASR hypotheses.
- We demonstrate through extensive experiments that our multi-view gated model outperforms single-view LSTM baselines, a fine-tuned mBART-50-large transformer baseline, and a non-gated multi-view concatenation model.
- We provide a comparative analysis against powerful Large Language Models, showing that our specialized, compact model is more effective for this targeted correction task.

2 Related Work

In recent years, there have been works on post-ASR correction. Li et al. [11] fine-tuned a multilingual LLM for post-ASR correction across multiple languages (including Hindi and Nepali in Devanagari), observing script-level transfer benefits for low-resource languages. Dutta et al. [4] introduce ROBART, a fine-tuned BART Seq2Seq model for ASR error correction in English, specifically targeting accented speech. Their approach leverages phoneme sequences as additional input and augmented training data generated from synthetic and real ASR errors, effectively reducing WER. Further improvements are achieved by combining ROBART’s output with ASR hypotheses using ROVER [5]. Weng et al. [22] designed a **Word-Confusion Pointer Network (WCN-Ptr)** with multi-head attention that jointly performs ASR correction and language understanding, achieving

about 14% relative error reduction. Shen et al. [21] introduced *MaskCorrect*, a training strategy that masks correct tokens to prevent trivial copying by transformer models for ASR correction in Mandarin.

Since our approach for ASR post-correction relies on a model that takes as input the ASR transcribed text, we also review work on OCR error correction in the Devanagari script. Post-OCR correction for Devanagari text has relied on character-level approaches in resource-constrained settings. Saluja et al. [19] introduced a character-level LSTM with fixed delay for OCR correction in Sanskrit and Hindi, achieving over 92% F_1 and about 26.7% WER reduction. Rijhwani et al. [17] proposed a multi-view, dual-encoder LSTM model for endangered languages, including Yakkha in Devanagari script, yielding about 34% error reduction. They later presented a semi-supervised approach [18] using lexically aware **weighted finite-state automata (WFSA)** decoding, achieving an additional 15–29% reduction in CER/WER without additional labels. Maheshwari et al. [13] released a Sanskrit post-OCR correction benchmark and dataset, showing that a byte-level model (ByT5) with phonetic encoding (SLP1) achieves about 23% improvement in error rates over raw OCR. Earlier, Krishna et al. [10] used a CopyNet Seq2Seq model with copying mechanisms on Romanized Sanskrit, boosting character recognition rate (CRR) from 35.8% to 87.0%. Meanwhile, Pal and Mustafi [14] combine BERT with Levenshtein distance for context-sensitive correction on Hindi OCR of the *Ramayana*, improving accuracy to about 81%.

3 Methodology

Our proposed method for post-ASR correction is a multi-view, character-level Seq2Seq model. This section details the problem formulation, our input alignment strategy, and the model architecture, with a focus on the gated fusion mechanism.

3.1 Problem Formulation

Given an audio utterance, we first obtain two distinct ASR hypotheses from our source models: the first one from Whisper denoted as $\mathbf{x}^{(1)} = [x_1^{(1)}, \dots, x_n^{(1)}]$ and the second one from MMS denoted as $\mathbf{x}^{(2)} = [x_1^{(2)}, \dots, x_m^{(2)}]$, where n and m are the lengths of the respective character sequences. The goal of our post-correction model is to learn a mapping from these two noisy inputs to the ground-truth transcription $\mathbf{y} = [y_1, \dots, y_K]$. We model this as finding the most likely corrected sequence $\hat{\mathbf{y}}$:

$$\hat{\mathbf{y}} = \arg \max_{\mathbf{y}'} p(\mathbf{y}' | \mathbf{x}^{(1)}, \mathbf{x}^{(2)}). \quad (1)$$

3.2 Input Alignment and Preprocessing

Since the two ASR hypotheses $\mathbf{x}^{(1)}$ and $\mathbf{x}^{(2)}$ can have different lengths and structures, a crucial first step is to align them. We employ a standard dynamic programming approach based on the Levenshtein distance to find the minimum edit distance alignment between the two character sequences. This process introduces a special alignment symbol ($\emptyset$) to equalize the lengths of the sequences, resulting in two aligned sequences of length T :

$$(\tilde{\mathbf{x}}^{(1)}, \tilde{\mathbf{x}}^{(2)}) = \text{Align}(\mathbf{x}^{(1)}, \mathbf{x}^{(2)}) \in (\Sigma \cup \{\emptyset\})^T \times (\Sigma \cup \{\emptyset\})^T, \quad (2)$$

where Σ is the character vocabulary. This alignment ensures a one-to-one correspondence between the characters of the two views at each time step t .

Motivation for multi-view approach. No single recognizer is consistently the best for Rajasthani. Whisper and MMS are trained with different data and modeling choices, so they make different kinds of mistakes. In practice, this means one hypothesis often preserves the right consonant sequence while the other preserves a local orthographic cue, such as nasalization or the length of a

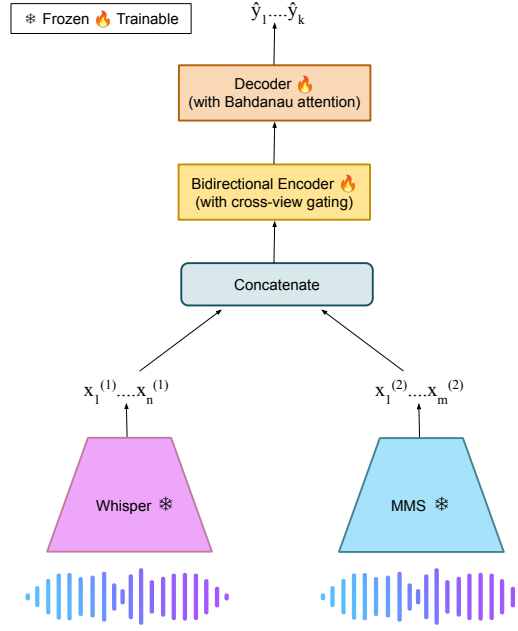


Fig. 1. The proposed multi-view architecture with character-wise gated fusion. The two aligned ASR hypotheses are embedded, concatenated, and passed through a single BiLSTM; a cross-view gating block then fuses information before decoding.

vowel, or it keeps a word boundary that the other system merges. When we expose the correction model to both hypotheses at once, it can choose the better view at each position and is more likely to recover the target form than any single-view correction model.

Whisper: धौलपुरे माय जिला निर्वाचन अधिकारी सगड़ा समाज रा लोगा सुं मतदान करबा खातर अपील करी है।

MMS: धौलपुरे माएँ जिला निर्वाचन अधिकारी सगड़ा समाजरा लोगा सुँ मतदान करबा खातर अपील करी है।

Ours: धौलपुर रै मांय जिला निर्वाचन अधिकारी सगळा समाज रा लोगा सुं मतदान करबा खातर अपील, करी है

Reference: धौलपुर रै मांय जिला निर्वाचन अधिकारी सगळा समाज रा लोगा सुं मतदान करबा खातर अपील, करी है

In this example, the model keeps the consonant sequence for “मांय” from Whisper and the nasalization from MMS, and resolves “सुं”/“सूँ” to “सूं” by keeping the long vowel from MMS and a stable nasal from Whisper, restores the split “समाज रा”, and corrects punctuation. These complementary strengths motivate taking both Whisper and MMS outputs as parallel inputs and letting the model arbitrate between them.

3.3 Model Architecture

We use an encoder–decoder architecture with an attention mechanism, illustrated in Figure 1. Given aligned character sequences ($\tilde{\mathbf{x}}^{(1)}, \tilde{\mathbf{x}}^{(2)}$), the pipeline comprises the following steps: (1) Embed each view and concatenate embeddings at each time step; (2) the concatenated stream is processed by a single BiLSTM [7]; (3) Refine the two view streams from the BiLSTM output using cross view gated fusion; (4) Predict the corrected characters using an additive attention decoder; (5) Use cross-entropy loss to perform end to end training.

Shared encoder (single BiLSTM over concatenated embeddings). At time t , we embed the aligned symbols from the two ASR outputs and concatenate them:

$$\mathbf{e}_t^{(1)} = \text{Emb}(\tilde{x}_t^{(1)}), \quad \mathbf{e}_t^{(2)} = \text{Emb}(\tilde{x}_t^{(2)}), \quad \mathbf{e}_t = [\mathbf{e}_t^{(1)}; \mathbf{e}_t^{(2)}] \in \mathbb{R}^{2E}.$$

A BiLSTM takes as input $\mathbf{e}_{1:T}$ and produces $\mathbf{h}_t \in \mathbb{R}^{2H}$:

$$H = \mathbf{h}_{1:T} = \text{BiLSTM}(\mathbf{e}_{1:T}).$$

This encoder is shared across all inputs (both views go through the *same* BiLSTM via concatenation).

Gated Fusion Mechanism. Since the encoder yields a single vector $\mathbf{h}_t \in \mathbb{R}^{2H}$ per step, we obtain *two streams* by **learned projections**:

$$\mathbf{v}_t^{(1)} = \underset{\in \mathbb{R}^{H \times 2H}}{P_1} \mathbf{h}_t, \quad \mathbf{v}_t^{(2)} = \underset{\in \mathbb{R}^{H \times 2H}}{P_2} \mathbf{h}_t,$$

yielding $\mathbf{v}_t^{(1)}, \mathbf{v}_t^{(2)} \in \mathbb{R}^H$. Cross-view gates $\mathbf{g}_{1,2}, \mathbf{g}_{2,1}$ then modulate residual exchanges:

$$\mathbf{g}_{12,t} = \sigma(W_{12} [\mathbf{v}_t^{(1)}; \mathbf{v}_t^{(2)}]) \in \mathbb{R}^H, \quad (3)$$

$$\hat{\mathbf{h}}_t^{(1)} = \mathbf{v}_t^{(1)} + \mathbf{g}_{12,t} \odot \tanh(\mathbf{v}_t^{(2)}), \quad (4)$$

$$\mathbf{g}_{21,t} = \sigma(W_{21} [\mathbf{v}_t^{(2)}; \mathbf{v}_t^{(1)}]) \in \mathbb{R}^H, \quad (5)$$

$$\hat{\mathbf{h}}_t^{(2)} = \mathbf{v}_t^{(2)} + \mathbf{g}_{21,t} \odot \tanh(\mathbf{v}_t^{(1)}), \quad (6)$$

$$\tilde{\mathbf{h}}_t = [\hat{\mathbf{h}}_t^{(1)}; \hat{\mathbf{h}}_t^{(2)}] \in \mathbb{R}^{2H}. \quad (7)$$

Here, P_1, P_2, W_{12}, W_{21} are learned linear layers; σ denotes the sigmoid and $\odot$ denotes element-wise product. When the gating block is disabled, we simply set $\tilde{\mathbf{h}}_t = \mathbf{h}_t$ (dual-channel concat without cross-view fusion).

Alignment symbol. We include the alignment marker $\emptyset$ in the source vocabulary as a normal token. During decoder attention, we apply a padding mask that hides only batch <pad> positions; $\emptyset$ remains visible. This allows the model to decide, through attention and gates, whether an aligned gap should be ignored or propagated.

Decoder with attention. The decoder is a unidirectional LSTM [8]. At step k , we use Bahdanau-style additive attention [1] to compute a context $\mathbf{c}_k$ over $\tilde{\mathbf{h}}_{1:T}$; the next character $\hat{y}_k$ is predicted from the decoder state and $\mathbf{c}_k$. Training minimizes token-level cross-entropy with teacher forcing. Inference uses greedy decoding.

4 Experiments

This section details the dataset and first-pass ASR setup (Section 4.1), the models we compare: single-view LSTM baselines, a single-view transformer baseline (mBART-50-large), our multi-view fusion models, and zero-shot LLMs (Section 4.2)—and the implementation details used for training and evaluation (Section 4.3).

4.1 Dataset and First-Pass ASR

We use the IndicTTS [3] dataset for Rajasthani available on the Hugging Face platform, which contains 9,060 audio samples. We create our own standardized splits, partitioning the data into **8,060** samples for training, **500** for validation, and **500** for testing. The splits are speaker-independent to ensure a fair evaluation. For each audio sample, we produce two first-pass hypotheses: one by transcribing with Whisper (large-v3) and another with MMS (1B-All). Both systems are used in

their standard transcription mode. The resulting pairs of (ASR hypothesis, ground-truth text) serve as inputs to our correction models and form the basis of our training and evaluation data.

4.2 Models for Comparison

We compare four different types of models: (i) ASR models, (ii) single-view Seq2Seq baselines, (iii) multi-view fusion models, and (iv) zero-shot/finetuned **Large Language Models (LLMs)**. All LSTM models share the same encoder/decoder architecture for a fair comparison.

ASR Models. We generate first-pass hypotheses by transcribing each audio clip in the dataset with Whisper-large-v3 and MMS-1B-All. The outputs are stored alongside their corresponding ground-truth transcriptions to form the training and evaluation sets. We report the baseline CER and WER for both models, providing a measure of initial ASR performance and the scope for improvement through post-correction.

Single-View Seq2Seq LSTM Baselines. To directly address the errors present in these first-pass hypotheses, we first consider single-view correction models trained on one *ASR source* at a time. Specifically, we identify LSTM (Whisper) and LSTM (MMS) as the standard character-level attentional encoder–decoder models trained separately on hypotheses from Whisper-large-v3 and MMS-1B-All.

To examine whether explicit copying improves character-level correction, we also include a pointer–generator variant [20] with an integrated copy mechanism, denoted LSTM+Copy (Whisper/MMS). At decoding step k , this model interpolates between generating from the character vocabulary and copying from the attended input:

$$p_{\text{gen}} = \sigma(\mathbf{w}_g^\top [\mathbf{s}_k; \mathbf{c}_k; \mathbf{y}_{k-1}] + b_g), \quad (8)$$

$$P(y | \cdot) = p_{\text{gen}} \cdot P_{\text{vocab}}(y) + (1 - p_{\text{gen}}) \cdot \sum_{t: x_t=y} \alpha_{k,t}, \quad (9)$$

where $\mathbf{s}_k$ is the decoder state, $\mathbf{c}_k$ is the attention context over encoder states, $\mathbf{y}_{k-1}$ is the previous output embedding, and $\alpha_{k,t}$ are attention weights over source positions. Here, $p_{\text{gen}} \in [0, 1]$ is the *generation probability* (the mixture gate) that trades off between generating and copying, and $P_{\text{vocab}}(y)$ is the decoder’s softmax distribution over the fixed character vocabulary; the copy term sums attention mass over all source positions t whose input character x_t matches the potential output character y .

Single-view Transformer Baseline (mBART-50-large). We fine-tune two mBART-50-large models [12]: one trained to correct Whisper hypotheses and one to correct MMS hypotheses, each framed as a single-view sequence correction task. As Rajasthani is not among the languages covered by mBART-50, this baseline tests whether multilingual denoising pretraining in Devanagari can transfer to Rajasthani via fine-tuning on a small dataset.

Multi-View Fusion Baselines. To leverage complementary information from both ASR hypotheses, we consider two fusion strategies applied after character-level alignment (Section 3). (1) Concat-LSTM forms a joint representation by concatenating the per-timestep embeddings from the aligned Whisper and MMS outputs, which are then processed by a shared BiLSTM encoder; the decoder attends over the resulting sequence without further interaction between the views. (2) Gated-LSTM (ours) augments this architecture with a lightweight, character-wise gated fusion module inserted between the encoder and attention layers as shown in Figure 1.

Fine-tuning/Zero-Shot LLMs. We fine-tune the Llama-3.2-3B model [6] on the train set. We also assess the capability of general-purpose large language models in a prompt-only, zero-shot setting:

GPT-4o [9] and Gemini 2.5 Pro [2]. We batch multiple sentences into a single prompt, instructing the models to correct the hypotheses generated by Whisper and MMS. The outputs are parsed line by line and evaluated similarly to other models.

4.3 Implementation Details

Our Seq2Seq LSTM models (single-view, concat, and gated) share a common architecture for fair comparison. The encoder is a 2-layer bidirectional LSTM, and the decoder is a 2-layer unidirectional LSTM, both with a hidden size of 256. Character embeddings are of size 128. We use the Adam optimizer with an initial learning rate of 10^{-3} and a batch size of 64. Models are trained for up to 100 epochs, while saving the best model based on the validation set CER. We fine-tune mBART-50-large with the following settings: learning rate 10^{-4} , 10 epochs, per-device train/eval batch size 5, gradient accumulation 100 steps (effective 500 examples per optimizer step), warmup 50 steps, evaluation every 50 steps. For Llama-3.2-3B the following settings were used: learning rate 10^{-5} , 3 epochs, per-device train/eval batch size 1, gradient accumulation 16 steps, warmup 50 steps, and evaluation is performed every epoch.

Features of our proposed multi-view architecture that help mitigate the risk of overfitting:

- **Validation-based model selection.** We train the model for a fixed number of epochs and save the checkpoint with the lowest validation CER.
- **Stochastic regularization.** Both the encoder and the decoder use dropout-based regularization with a dropout rate of 0.2. The optimizer applies gradient-norm clipping ($\|g\| \leq 1$) to prevent unstable updates that amplify memorization.
- **Padding-aware training (less noise, stable optimization).** We bucket batches by length and use packed sequences with padding masks so the loss excludes padding tokens. This reduces spurious gradients and makes optimization more sample-efficient on small corpora.
- **Lightweight architecture.** The correction model is a small, character-level 2-layer BiLSTM→LSTM with attention (128-dim embeddings; 256/512 hidden). Its capacity is quite limited compared with large word-piece models.

5 Results and Analysis

We evaluate all models on the held-out Rajasthani test set and report both CER and WER. Results are presented for raw ASR outputs, single-view and multi-view correction models, and zero-shot LLMs. The analysis considers both absolute error rates and relative improvements over competitive baselines, while also highlighting each approach's ability to reduce insertion, deletion, and substitution errors and to preserve correct word boundaries.

5.1 Overall Performance

Table 1 reports the CER and WER of all the models on the Rajasthani test set. Among all models, our Gated-LSTM achieves the lowest error rates, with a CER of 7.86% and a WER of 29.98%. Compared with the best single-view baseline (8.01% CER, 30.33% WER), this corresponds to a relative reduction of approximately 1.87% in CER and 1.15% in WER. Against the simpler multi-view concatenation approach (8.21% CER, 30.05% WER), the gating mechanism yields a further 4.26% relative CER improvement and 0.23% relative WER improvement.

When compared with the raw ASR hypotheses, Gated-LSTM delivers substantial gains. Using Whisper output as input, CER decreases from 16.60% to 7.86% (a 52.65% relative reduction), and WER drops from 62.30% to 29.98% (a 51.90% reduction). Using MMS output, CER is reduced from 15.73% to 7.86% (a 50.02% reduction) and WER from 60.24% to 29.98% (a 50.25% reduction).

Table 1. Performance Evaluation on the Rajasthani Test Set (CER and WER, Lower is Better). “Input” Denotes the Source Text Given to The Model: IndicTTS Audio for Raw ASR Systems, Recognized Text from the Specified ASR for Single-View and LLM Models, or Aligned Hypotheses from Both ASRs for Multi-View Models

Model	Input	CER (%)	WER (%)	Inference time (ms/sample)
Raw ASR outputs				
Whisper-large-v3 (1-best)	IndicTTS audio	16.60	62.30	547.16
MMS-1B-All (1-best)	IndicTTS audio	15.73	60.24	37.61
Whisper-large-v3 (1-best) (finetuned)	IndicTTS audio	17.11	38.05	551.94
MMS-1B-All (1-best) (finetuned)	IndicTTS audio	8.73	36.55	43.58
Single-view baselines				
LSTM	Whisper recognized text	8.28	30.66	3.47
LSTM	MMS recognized text	8.01	30.33	2.40
LSTM+Copy	Whisper recognized text	8.64	31.40	2.65
LSTM+Copy	MMS recognized text	8.97	31.33	2.54
Single-view finetuned				
Llama-3.2-3B	Whisper recognized text	11.47	40.83	893.4
Llama-3.2-3B	MMS recognized text	11.30	40.18	846.1
mBART-50-large	Whisper recognized text	18.60	66.23	311.84
mBART-50-large	MMS recognized text	18.15	69.11	306.61
Multi-view fusion				
Concat-LSTM	Whisper + MMS recognized text	8.21	30.05	5.92
Gated-LSTM (Ours)	Whisper + MMS recognized text	7.86	29.98	6.40
Zero-shot LLMs				
GPT-4o	Whisper recognized text	19.83	59.48	-
GPT-4o	MMS recognized text	20.56	60.11	-
Gemini 2.5 Pro	Whisper recognized text	11.73	45.61	-
Gemini 2.5 Pro	MMS recognized text	10.55	41.86	-

We also fine-tune the ASR models. Fine-tuned Whisper attains 17.11% CER and 38.05% WER, reducing WER markedly relative to the raw Whisper output (62.30%), while the proposed correction model still delivers substantially lower CER 7.86% and WER 29.98%.

We finetuned Llama-3.2-3B in a single-view setting, achieving 11.47% CER and 40.83% WER when given Whisper hypotheses as input, and 11.30% CER and 40.18% WER when given MMS hypotheses as input. Despite fine-tuning, Llama-3.2-3B’s performance remains substantially behind our Gated-LSTM.

In contrast, the zero-shot LLM baselines perform inconsistently. While Gemini 2.5 Pro shows moderate improvement over raw ASR outputs, especially for MMS input (CER reduces from 15.73% to 10.55%, WER reduces from 60.24% to 41.86%), GPT-4o tends to underperform, often generating additional characters or words that limit overall error reduction.

Fine-tuned mBART-50-large trails both baselines: on Whisper input, it reaches 18.60% CER / 66.23% WER (vs. LSTM 8.28/30.66), and on MMS input 18.15% / 69.11% (vs. LSTM 8.01/30.33); both are far behind the multi-view gated model (7.86/29.98).

5.2 Computational Cost

The computational cost (Inference time) per data sample, where one sample corresponds to a single input utterance from the test set, is reported in Table 1. Our Gated-LSTM based correction model attains the best post-ASR accuracy while running at 6.40 ms/sample on A6000 GPU. This is $\sim 170\times$ faster than the finetuned Llama-3.2-3B (~ 890 ms/sample) and $\sim 49\times$ faster than fine-tuned mBART-50 (~ 310 ms/sample), yet achieves clearly lower error rates. The small gap to Concat-LSTM (5.92 ms/sample) reflects the gating overhead ($\sim 8\%$), which pays off as accuracy improvement. The End-to-End computational cost of the two-stage pipeline is dominated by the ASR models (Whisper ~ 550 ms/sample and MMS ~ 40 ms/sample), as our LSTM pipeline is very fast in comparison.

5.3 Ablation Studies

Concat vs. Gated Fusion. When both ASR hypotheses are aligned at the character level, the Concat-LSTM baseline achieves a modest improvement over LSTM with MMS input, the best performing single-view model (WER 30.05 vs. 30.33), confirming that the two ASR outputs contain complementary errors that can be exploited. Introducing our character-wise gated fusion mechanism yields an additional reduction to 7.86 CER and 29.98 WER, corresponding to a relative gain of 4.26% in CER and 0.23% in WER compared with concatenation. These results indicate that data-dependent weighting of the two views prior to decoding is more effective than fixed early fusion, enabling the model to selectively emphasize the correct signal at each character position. Computationally, the gating adds only a small latency overhead (6.40 vs. 5.92ms/sample; ~8%), which is amply compensated by the consistent accuracy gains.

Copy Mechanism (Pointer-Generator) vs. No-Copy. In the single-view setting, adding a pointer-generator copy mechanism consistently underperforms the vanilla architecture (e.g., CER 8.64/8.97 vs. 8.28/8.01 for Whisper/MMS inputs). We attribute this to the model's tendency to copy ASR-specific noise—such as near-miss spellings, redundant diacritics, or spurious insertions—rather than generating the correct form. Without copying, the decoder is forced to rely more heavily on learned orthographic and morphological regularities of Devanagari, thus improving robustness. Given this observation, we have implemented all multi-view models without using the copy mechanism.

Transformer Baseline (mBART-50-large). Despite multilingual pretraining, mBART-50-large performs considerably worse than our character-level LSTMs (18.60/66.23 on Whisper; 18.15/69.11 on MMS). We observe that unconstrained sequence generation likely encourages paraphrasing, which inflates WER; additional constraints or correction-specific objectives may be required to harness mBART effectively for Rajasthani post-ASR correction.

5.4 Finetuned/Zero-Shot LLMs

We feed the outputs of Whisper & MMS as input to GPT-4o and Gemini 2.5 pro. For Whisper input, GPT-4o results in CER rising from 16.60 to 19.83, WER reducing from 62.30 to 59.48. For MMS input, GPT-4o results in CER rising from 15.73 to 20.56, and WER reducing from 60.24 to 60.11. This suggests that unconstrained generation often leads to over-correction or paraphrasing that deviates from the target transcription. Gemini 2.5 Pro shows clear gains over raw ASR—achieving CER 11.73 (Whisper) and 10.55 (MMS) versus 16.60 and 15.73 for raw ASR, respectively—but remains substantially behind domain-specific models. Relative to our Gated-LSTM, Gemini's CER is 49% higher on Whisper inputs and 34% higher on MMS inputs, while its WER is 52% and 40% higher, respectively. This gap highlights the advantage of targeted training over zero-shot prompting for post-ASR correction in low-resource, dialect-rich settings.

We additionally experiment with finetuned Llama-3.2-3B in a single-view setup. On Whisper input, it attains 11.47% CER and 40.83% WER; on MMS input, it achieves 11.30% CER and 40.18% WER. Despite fine-tuning, WER remains ~34–36% higher and CER remains ~44–46% higher than our Gated-LSTM. Inference is also substantially slower (846/893 ms/sample), i.e., ~133–140× slower than the 6.40 ms/sample Gated-LSTM.

5.5 Error Analysis

We examine four error categories: **insertion errors (IE)**, **substitution errors (SE)**, **deletion errors (DE)**, and **word-boundary errors (WBE)**. IE/SE/DE are computed at both character and word levels; all white space-related errors are captured under WBE, which reflects merges or splits of adjacent words.

Table 2. Error Counts. MMS is the Baseline; Numbers in Parentheses Show % Change Relative to MMS. WBE is Computed at Boundary/Character Level

Model	Character/Boundary				Word		
	IE	SE	DE	WBE	IE	SE	DE
MMS (baseline)	980	3805	3594	1462	160	5004	974
Gemini-2.5-pro	1072 (-9.39%)	2580 (32.19%)	1967 (45.27%)	506 (65.39%)	311 (-94.38%)	3818 (23.70%)	137 (85.93%)
GPT-4o	2808 (-186.53%)	4770 (-25.36%)	3375 (6.09%)	1320 (9.71%)	581 (-263.12%)	5141 (-2.74%)	403 (58.62%)
Multi-view LSTM	1296 (-32.24%)	1590 (58.21%)	1261 (64.91%)	395 (72.98%)	188 (-17.50%)	2699 (46.06%)	133 (86.34%)

5.5.1 Quantitative Analysis. Table 2 reports absolute error counts on the test set and percentage change relative to the MMS baseline. Positive percentages indicate a reduction in errors; negative percentages indicate an increase.

Multi-view LSTM (Whisper+MMS inputs). At the character level, SE decrease by 58.21% and DE by 64.91%, and WBE falls by 72.98%, indicating substantially better boundary placement. At the word level, SE drops by 46.06% and DE drops by 86.34%. However, IE increases by 32.24% at the character level and by 17.50% at the word level. This rise in IEs is compensated by the large reductions in substitutions, deletions, and WBE.

Gemini-2.5 Pro (MMS input). Character-level SE and DE are reduced by 32.19% and 45.27%, and WBE is reduced by 65.39%. At the word level, SE and DE decline by 23.70% and 85.93%. IE, however, increase by 9.39% at the character level and by 94.38% at the word level, reflecting a tendency to introduce extra words.

GPT-4o (MMS input). IE increases markedly by 186.53% at the character level and 263.12% at the word level, and SE increases slightly by 25.36% and 2.74% at the character and word levels, respectively. DE are reduced by 6.09% at the character level and 58.62% at the word level, and WBE improves by 9.71%. Overall, the profile is dominated by added and rephrased content rather than targeted error correction.

Summarizing the quantitative analysis, we observe that the Multi-view LSTM provides the strongest overall reductions in SE, DE, and WBE with modest increases in insertions. Gemini-2.5 Pro yields moderate improvements but introduces additional words. GPT-4o predominantly increases IE and, to a lesser extent, SE, despite reductions in DE and a small gain in WBE.

5.5.2 Qualitative Analysis. We analyze two examples, sentences shown in Table 3, to illustrate typical model behaviors and their impact on substitution, insertion, deletion, and WBE.

Whisper. The output contains substitutions that replace dialectal Rajasthani forms with standardized Hindi equivalents, such as “सरवाती” for “सरुआती” and “वेरियो” for “व्हे रियो”. We observe several instances of words getting merged, for example “आयोगने” instead of “आयोग नै”, and “जारियो” instead of “जा रियो”. These patterns explain the high substitution counts and frequent boundary errors observed in the raw output.

MMS. MMS produces an error profile similar to Whisper, with dialectal forms replaced by standardized ones (“दिना” for “दिनां”) and merged auxiliaries (“लिख्यो जारियो” for “लिख्यो जा रियो”). While some word boundaries are preserved compared with Whisper, substitution and boundary errors remain prevalent.

Multi-view LSTM (Whisper+MMS inputs). Our proposed model corrects most dialectal substitutions and restores proper word boundaries. Examples include “दिनां” instead of “दिना”, “व्हे रियो” instead of “वेरियो”, and “नै” instead of “ने”. These changes directly reduce substitution counts and

Table 3. Example Outputs for Two Representative Sentences from the Evaluation Set, Showing the Ground Truth (GT), Raw ASR Hypotheses (Whisper, MMS), and Corrected Outputs from the Proposed Multi-View LSTM, Gemini-2.5-pro, and GPT-4o. Green Highlights Indicate Correct Words Matching the GT After Correction; Red Highlights Indicate Errors Such as Substitutions, Insertions, Deletions, or Incorrect Word Boundaries

Models	Output Sentences
GT	Sentence 1: सरुआती दिनां में लर्निंग लाइसेंस खातर, या काम सरु व्हे रियो है Sentence 2: इण वास्ते आयोग नै कागद लिख्यो जा रियो है
Whisper Output	Sentence 1: सरवाती दिना में लर्निंग लाइसेंस खातर या काम सरु वेरियो है। Sentence 2: इन वास्ते आयोगने कागद लिख्यो जारियो है।
MMS Output	Sentence 1: सरुआती दिना में लर्निंग लाइसेंस खातर या काम सरु वेरियो है Sentence 2: इन वास्ते आयोग ने कागद लिख्यो जारियो है
Multi-view LSTM (Proposed)	Sentence 1: सरुआती दिनां में लर्निंग लाइसेंस खातर, या काम सरु व्हे रियो है Sentence 2: इण वास्ते आयोग नै कागद लिख्यो जा रियो है
Gemini-2.5-pro (MMS Input)	Sentence 1: सरुआती दिना में लर्निंग लाइसेंस खातर या काम सरु वे रियो है Sentence 2: इन वास्ते आयोग ने कागद लिख्यो जा रियो है
GPT-4o (MMS Input)	Sentence 1: शुरुआती दिनां माई लर्निंग लाइसेंस खातर या काम शुरु करयो गयो है Sentence 2: इन खातर आयोग ने कागद लिखायो जारी है

WBE while keeping IE and DE minimal, indicating precise, local corrections rather than a large number of rewrites.

Gemini-2.5 Pro (MMS input). Gemini makes fewer changes than the proposed model and often retains MMS errors. For example, “दिना” remains uncorrected, and aspiration loss is seen in “वे रियो” (“व्ह” → “व”). Dialectal replacements such as “इण” for “इन” are missed. There is limited improvement in substitution and boundary metrics.

GPT-4o (MMS input). GPT-4o frequently rewrites sentences, introducing paraphrases and standardized Hindi forms. For example, “सरुआती” is replaced with “शुरुआती”, and clauses such as “लिख्यो जा रियो” become “लिखायो जारी”. These changes can increase SE counts and do not consistently resolve boundary errors, reflecting a tendency toward normalization rather than minimal error correction.

Across the examples, the proposed Multi-view LSTM achieves the most targeted corrections, addressing both dialectal substitutions and WBE without introducing new errors. Gemini-2.5 Pro makes smaller improvements, largely retaining MMS errors. GPT-4o demonstrates fluent generation but alters sentence structure and style in ways that increase measured errors.

5.6 Character-Level Alignment beyond Devanagari

We test the usability of our character-level alignment strategy beyond the Devanagari script. We conduct additional experiments on the Tamil script. We utilize the IndicTTS Tamil dataset, dividing it into 8437 training samples and 500 samples each for the validation and test sets. Results of experiments on the Tamil test set are shown in Table 4, where Gated-LSTM (multi-view) achieves 4.02 % CER, 19.78 % WER, improving upon the best single-view LSTM (4.07% CER, 20.68 % WER) with ~1.2 % relative CER and ~4.4 % relative WER reductions. These results indicate that our Character-level alignment can be effective on scripts other than Devanagari.

Table 4. Model Performances on Tamil Test Set

Model	Input	CER (%)	WER (%)
Raw ASR outputs			
Whisper-large-v3 (1-best)	IndicTTS audio	9.75	42.09
MMS-1B-All (1-best)	IndicTTS audio	7.73	44.57
LSTM Models			
LSTM	Whisper recognized text	4.83	24.79
LSTM	MMS recognized text	4.07	20.68
Gated-LSTM (Ours)	Whisper + MMS recognized text	4.02	19.78

6 Conclusion

In this article, we addressed the challenge of ASR for the low-resource Rajasthani language. Acknowledging that state-of-the-art multilingual ASR systems produce significant errors for this language, we proposed a practical post-ASR correction framework. Our approach leverages the noisy outputs of two powerful ASR models, Whisper and MMS, within a multi-view Seq2Seq architecture. Our key contribution is a character-wise gated fusion mechanism that learns to combine information from two ASR views. On IndicTTS Rajasthani, the Gated-LSTM attains the lowest error rates, outperforming both single-view baselines and a non-gated multi-view concatenation. It also outperforms the large general-purpose LLMs in zero-shot (GPT-4o and Gemini 2.5 Pro) and fine-tuning (Llama-3.2-3B) mode, underscoring the value of task-specific training and inductive bias in low-resource settings. Finally, a transformer baseline based on mBART-50-large fine-tuned separately on Whisper and MMS hypotheses lags well behind the LSTM models, suggesting that multilingual denoising pretraining in Devanagari alone does not transfer effectively to post-ASR correction under small-data fine-tuning. Our method is computationally efficient while achieving the best CER and WER. Experiments on Tamil show that the method can be applied to scripts other than Devanagari.

This work establishes the first benchmark for post-ASR correction in Rajasthani. Future work could explore more sophisticated fusion mechanisms and apply this multi-view correction framework to other low-resource Indic languages.

References

- [1] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2014. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473* (2014).
- [2] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Naveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261* (2025).
- [3] Speech Technology Consortium, Hema A Murthy, and S Umesh. 2023. Indic TTS: A Text-to-Speech Database for Indian Languages. Retrieved from <https://www.iitm.ac.in/donlab/indicetts/>
- [4] Samrat Dutta, Shreyansh Jain, Ayush Maheshwari, Souvik Pal, Ganesh Ramakrishnan, and Preethi Jyothi. 2022. Error correction in asr using sequence-to-sequence models. *arXiv preprint arXiv:2202.01157* (2022).
- [5] Jonathan G Fiscus. 1997. A post-processing system to yield reduced word error rates: Recognizer output voting error reduction (ROVER). In *1997 IEEE Workshop on Automatic Speech Recognition and Understanding Proceedings*. IEEE, 347–354.
- [6] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [7] Alex Graves and Jürgen Schmidhuber. 2005. Framewise phoneme classification with bidirectional LSTM and other neural network architectures. *Neural Networks* 18, 5-6 (2005), 602–610.
- [8] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural Computation* 9, 8 (1997), 1735–1780.

- [9] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276* (2024).
- [10] Amrith Krishna, Bodhisattwa Prasad Majumder, Rajesh Shreedhar Bhat, and Pawan Goyal. 2018. Upcycle your OCR: Reusing OCRs for post-OCR text correction in romanised sanskrit. *arXiv preprint arXiv:1809.02147* (2018).
- [11] Sheng Li, Chen Chen, Chin Yuen Kwok, Chenhui Chu, Eng Siong Chng, and Hisashi Kawai. 2024. Investigating asr error correction with large language model and multilingual 1-best hypotheses. In *Proc. Interspeech*, Vol. 2024. 1315–1319.
- [12] Yinhan Liu, Jiatao Gu, Naman Goyal, Xian Li, Sergey Edunov, Marjan Ghazvininejad, Mike Lewis, and Luke Zettlemoyer. 2020. Multilingual denoising pre-training for neural machine translation. *Transactions of the Association for Computational Linguistics* 8 (2020), 726–742.
- [13] Ayush Maheshwari, Nikhil Singh, Amrith Krishna, and Ganesh Ramakrishnan. 2022. A benchmark and dataset for post-OCR text correction in Sanskrit. *arXiv preprint arXiv:2211.07980* (2022).
- [14] Aditya Pal and Abhijit Mustafi. 2020. Vartani spellcheck—automatic context-sensitive spelling correction of OCR-generated hindi text using BERT and levenshtein distance. *arXiv preprint arXiv:2012.07652* (2020).
- [15] Vineel Pratap, Andros Tjandra, Bowen Shi, Paden Tomasello, Arun Babu, Sayani Kundu, Ali Elkahky, Zhaoheng Ni, Apoorv Vyas, Maryam Fazel-Zarandi, Alexei Baevski, Yossi Adi, Xiaohui Zhang, Wei-Ning Hsu, Alexis Conneau, and Michael Auli. 2023. Scaling speech technology to 1,000+ languages. *arXiv* (2023).
- [16] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. 2022. Robust Speech Recognition via Large-Scale Weak Supervision. DOI : <https://doi.org/10.48550/ARXIV.2212.04356>
- [17] Shruti Rijhwani, Antonios Anastasopoulos, and Graham Neubig. 2020. OCR post correction for endangered language texts. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 5931–5942.
- [18] Shruti Rijhwani, Daisy Rosenblum, Antonios Anastasopoulos, and Graham Neubig. 2021. Lexically aware semi-supervised learning for OCR post-correction. *Transactions of the Association for Computational Linguistics* 9 (2021), 1285–1302.
- [19] Rohit Saluja, Devaraj Adiga, Parag Chaudhuri, Ganesh Ramakrishnan, and Mark Carman. 2017. Error detection and corrections in Indic OCR using LSTMs. In *2017 14th IAPR International Conference on Document Analysis and Recognition (ICDAR)*, Vol. 1. IEEE, 17–22.
- [20] Abigail See, Peter J Liu, and Christopher D Manning. 2017. Get to the point: Summarization with pointer-generator networks. *arXiv preprint arXiv:1704.04368* (2017).
- [21] Kai Shen, Yichong Leng, Xu Tan, Siliang Tang, Yuan Zhang, Wenjie Liu, and Edward Lin. 2022. Mask the correct tokens: An embarrassingly simple approach for error correction. *arXiv preprint arXiv:2211.13252* (2022).
- [22] Yue Weng, Sai Sumanth Miryala, Chandra Khatri, Runze Wang, Huaixiu Zheng, Piero Molino, Mahdi Namazifar, Alexandros Papangelis, Hugh Williams, Franziska Bell, et al. 2020. Joint contextual modeling for asr correction and language understanding. In *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 6349–6353.

Received 22 August 2025; revised 16 November 2025; accepted 17 January 2026